

1. Introduction to AJAX

What is AJAX?

AJAX stands for

Asynchronous **JavaScript** And **XML**

Nowadays, AJAX mostly exchanges **JSON** instead of XML.

AJAX allows a web page to

- Update data without refreshing the page
- Send requests to a server
- Receive data from a server
- Create fast and interactive web applications

Without AJAX

User

|

V

Browser

|

Entire Page Request

|

Server

|

Entire Page Reload

Every click reloads the webpage.

With AJAX

User

|

V

Browser

|

AJAX Request

|

Server

|

JSON Response

|

Only Required Data Updates

The page does not reload.

Advantages

- Faster websites
- No page refresh
- Less bandwidth
- Better user experience

Applications

- Gmail
- Google Maps
- Facebook Notifications
- Instagram Feed
- Online Shopping

2. Working of AJAX

Steps

User Clicks Button
|
JavaScript sends Request
|
Server receives Request
|
Server processes Data
|
Server returns JSON
|
JavaScript updates HTML

3. XMLHttpRequest Object

Before Fetch API, AJAX used

XMLHttpRequest

Important Methods

Method	Purpose
open()	Opens request
send()	Sends request
onreadystatechange	Detects response
status	HTTP Status
responseText	Server response

XMLHttpRequest Syntax

```
let xhr = new XMLHttpRequest();  
  
xhr.open("GET", "data.json", true);  
  
xhr.send();
```

4. AJAX Example using XMLHttpRequest

Create three files.

index.html

```
<!DOCTYPE html>  
<html>  
<head>  
<title>AJAX Example</title>  
</head>  
<body>  
  
<h2>AJAX Example</h2>  
  
<button onclick="loadData()">  
Load Data  
</button>  
  
<p id="demo"></p>  
  
<script src="script.js"></script>
```

```
</body>
</html>
```

data.json

```
{
  "name":"CBIT",
  "course":"Full Stack Development",
  "year":"Second Year"
}
```

script.js

```
function loadData(){

let xhr = new XMLHttpRequest();

xhr.onreadystatechange=function(){

if(this.readyState==4 && this.status==200){

let obj=JSON.parse(this.responseText);

document.getElementById("demo").innerHTML=
obj.name+"<br>"+
obj.course+"<br>"+
obj.year;

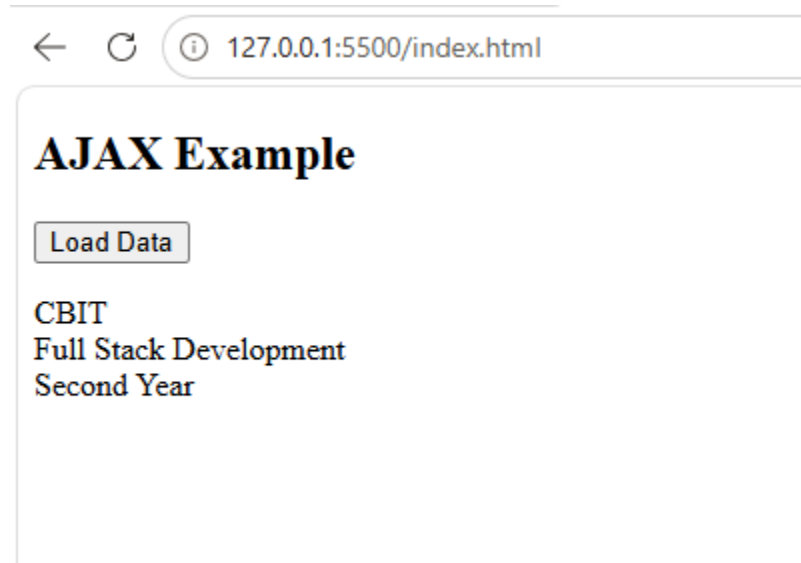
}

};

xhr.open("GET","data.json",true);

xhr.send();

}
```



Output

AJAX Example

[Load Data]

CBIT
Full Stack Development
Second Year

XMLHttpRequest Flow

Create XMLHttpRequest

↓

open()

↓

send()

↓

Server Response

↓

responseText

↓

Update HTML

5. Fetch API

Modern JavaScript uses

fetch()
instead of XMLHttpRequest.

Advantages

- Cleaner syntax
- Promise-based
- Easier to understand
- Better error handling

Syntax

```
fetch("data.json")  
.then(response=>response.json())  
.then(data=>console.log(data));
```

6. Fetch API Example

index.html

```
<!DOCTYPE html>  
<html>
```

```
<head>
<title>Fetch API Example</title>
</head>
<body>

<h2>Fetch API</h2>

<button onclick="getData()">
Get Data
</button>

<p id="demo"></p>

<script>

function getData(){

fetch("data.json")

.then(response=>response.json())

.then(data=>{

document.getElementById("demo").innerHTML=

data.name+"<br>" +

data.course+"<br>" +

data.year;

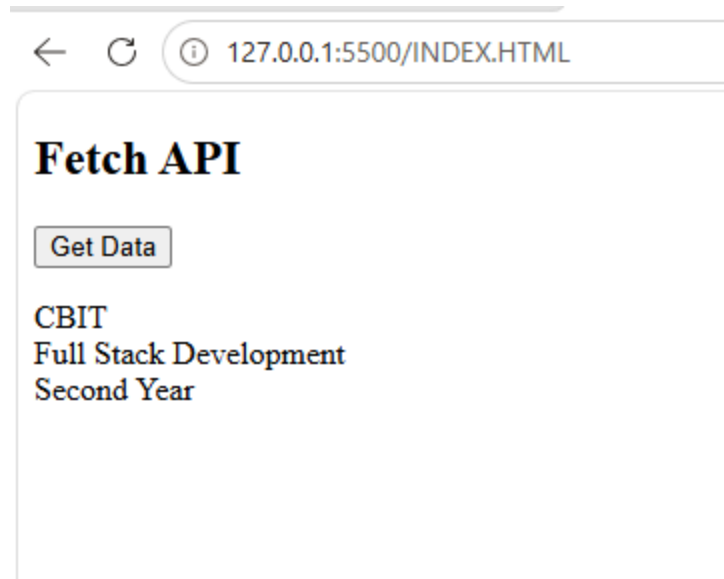
})

.catch(error=>console.log(error));

}

</script>

</body>
</html>
```



Output

CBIT

Full Stack Development

Second Year

7. JSON

JSON stands for

JavaScript Object Notation

Used to exchange data between

- Browser
- Server

JSON Example

```
{  
  "name": "CBIT",  
  "course": "MERN",  
  "year": 2  
}
```

JavaScript Object

```
let student={  
  name:"CBIT",  
  course:"MERN",  
  year:2  
};
```

Differences

JSON	JavaScript Object
Keys inside quotes	Quotes optional
String	Object
Used for data transfer	Used inside JavaScript

8. JSON Methods

JSON.parse()

Converts

JSON

↓

JavaScript Object

Example

Method 1: Using Visual Studio Code (Recommended)

Step 1: Create a file named index.html

```
<!DOCTYPE html>  
<html>  
<head>
```

```
<title>JSON.parse() Example</title>
</head>
<body>

<h2>JSON.parse() Example</h2>

<p id="demo"></p>

<script>

let text = '{"name":"CBIT"}';

let obj = JSON.parse(text);

console.log(obj.name);

// Display on webpage
document.getElementById("demo").innerHTML = obj.name;

</script>

</body>
</html>
```

Step 2

Save the file.

Step 3

Install **Live Server** in VS Code.

Step 4

Right-click on index.html



Open with Live Server

Output on Webpage



JSON.parse() Example

CBIT

Console Output

```
let text='{"name":"CBIT"}';
```

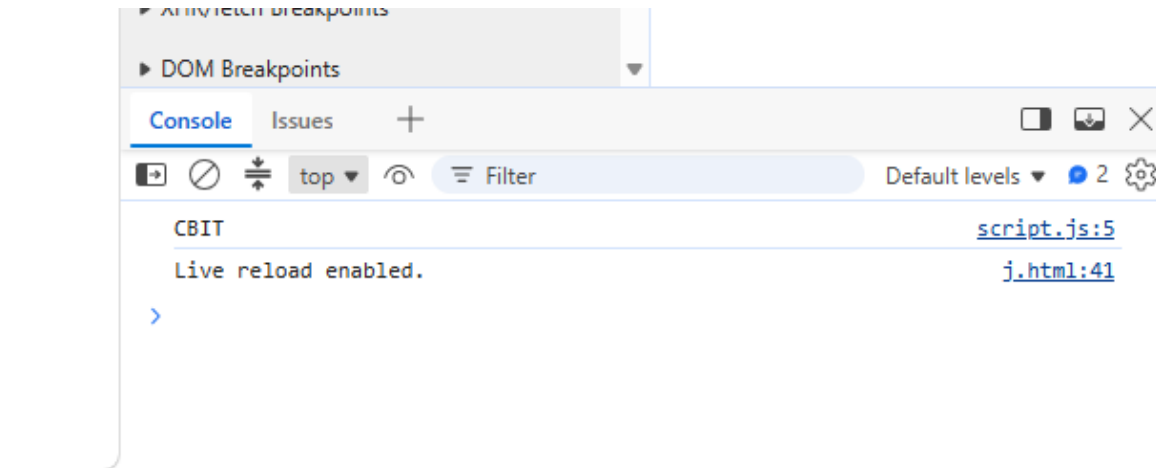
```
let obj=JSON.parse(text);
```

```
console.log(obj.name);
```

Output

CBIT





JSON.stringify()

Converts

Object

↓

JSON

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>JSON.stringify() Example</title>

</head>

<body>

<h2>JSON.stringify() Example</h2>

<button onclick="convertJSON()">Convert to JSON</button>

<h3>Output</h3>

<p id="object"></p>

<p id="json"></p>

<script>

```
function convertJSON() {
```

```
    // JavaScript Object
```

```
    let student = {
```

```
        name: "CBIT",
```

```
        course: "Full Stack Development",
```

```
        year: 2,
```

```
        city: "Hyderabad"
```

```
    };
```

```
    // Display JavaScript Object
```

```
    document.getElementById("object").innerHTML =
```

```
        "<b>JavaScript Object:</b><br>" +
```

```
        "Name: " + student.name + "<br>" +
```

```
        "Course: " + student.course + "<br>" +
```

```
        "Year: " + student.year + "<br>" +
```

```
        "City: " + student.city;
```

```
// Convert Object into JSON String
```

```
let jsonData = JSON.stringify(student);
```

```
// Display JSON String
```

```
document.getElementById("json").innerHTML =
```

```
"<br><b>JSON String:</b><br>" +
```

```
jsonData;
```

```
// Console Output
```

```
console.log("JavaScript Object:");
```

```
console.log(student);
```

```
console.log("JSON String:");
```

```
console.log(jsonData);
```

```
}
```

```
</script>
```

```
</body>
```

```
</html>
```

← ↻ ⓘ 127.0.0.1:5500/jsonstring.html

JSON.stringify() Example

Convert to JSON

Output

← ↻ ⓘ 127.0.0.1:5500/jsonstring.html

JSON.stringify() Example

Convert to JSON

Output

JavaScript Object:

Name: CBIT

Course: Full Stack Development

Year: 2

City: Hyderabad

JSON String:

```
{"name":"CBIT","course":"Full Stack Development","year":2,"city":"Hyderabad"}
```

9. Sending JSON Data

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Sending JSON</title>
```

```
</head>
```

```
<body>
```

```
<h2>Sending JSON Data</h2>
```

```
<button onclick="sendData()">Send Data</button>
```

```
<script>
```

```
function sendData(){
```

```
  let student={
```

```
    name:"CBIT",
```

```
    course:"MERN",
```

```
    year:2
```

```
  };
```

```
  let jsonData=JSON.stringify(student);
```

```
  console.log(jsonData);
```

```
  alert(jsonData);
```

```
}
```

```
</script>
```

</body>

</html>



10. Receiving JSON Data

Receiving JSON Data Asynchronously (Fetch API)

Folder Structure

AJAX

index.html

data.json

data.json

```
{  
  "name":"CBIT",  
  "course":"Full Stack Development",  
  "year":2  
}
```

index.html

```
<!DOCTYPE html>  
<html>  
  <head>  
    <title>Receive JSON</title>  
  </head>  
  <body>
```

```
<h2>Receiving JSON Data</h2>
```

```
<button onclick="loadData()">Load Data</button>
```

```
<p id="demo"></p>
```

```
<script>
```

```
function loadData(){
```

```
  fetch("data.json")
```

```
  .then(response=>response.json())
```

```
  .then(data=>{
```

```
    document.getElementById("demo").innerHTML=
```

```
    data.name+"<br>" +
```

```
    data.course+"<br>" +
```

```
    data.year;
```

```
  });
```

```
}
```

```
</script>
```

```
</body>
```

```
</html>
```

Output

CBIT

Full Stack Development

2

Flow

Button Click

↓

fetch()

↓

Read data.json

↓

Convert to Object

↓

Display on Screen



11. Handling Server Responses

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Server Response</title>
```

```
</head>
```

```
<body>
```

```
<h2>Handling Server Responses</h2>
```

```
<button onclick="getData()">Get Data</button>
```

```
<p id="demo"></p>
```

```
<script>
```

```
function getData(){
```

```
  fetch("data.json")
```

```
    .then(response=>{
```

```
      if(response.ok){
```

```
        return response.json();
```

```
      }
```

```
    else{
```

```
      throw new Error("File Not Found");
```

```
    }
```

```
  })
```

```
    .then(data=>{
```

```
      document.getElementById("demo").innerHTML=data.name;// to display only name
```

```
document.getElementById("demo").innerHTML =  
    "<b>Name:</b> " + data.name + "<br>" +  
    "<b>Course:</b> " + data.course + "<br>" +  
    "<b>Year:</b> " + data.year;  
  
    })  
  
    .catch(error=>{  
  
        document.getElementById("demo").innerHTML=error;  
  
        });  
  
    }  
  
</script>  
  
</body>  
</html>
```

Output



Network Error
if the file is missing.(HANDLING ERRORS)

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Handling Server Responses</title>
```

```
</head>
```

```
<body>
```

```
<h2>Handling Server Responses using Fetch API</h2>
```

```
<button onclick="getData()">Load Data</button>
```

```
<h3>Output</h3>
```

```
<p id="demo"></p>
```

```
<script>
```

```
function getData() {
```

```
    fetch("data.json")
```

```
    .then(function(response) {
```

```
        // Check whether the response is successful
```

```
        if (response.ok) {
```

```
            return response.json();
```

```
        }
```

```
        throw new Error("Unable to load data!");
```

```
    })
```

```
    .then(function(data) {
```

```
        document.getElementById("demo").innerHTML =
```

```
        "<b>Name:</b> " + data.name + "<br>" +
```

```
        "<b>Course:</b> " + data.course + "<br>" +
```

```
        "<b>Year:</b> " + data.year;
```

```
    })
```

```
    .catch(function(error) {
```

```
document.getElementById("demo").innerHTML =  
    "<span style='color:red'>" + error.message + "</span>";  
  
});  
  
}  
  
</script>  
  
</body>  
</html>
```



RENAME THE FILE FROM “DATA.JSON” TO “DEMO.JSON”

RUN THE PROGRAM AGAIN



Receiving JSON using XMLHttpRequest

data.json

```
{  
  "name": "CBIT",  
  "course": "FSD"  
}
```

Click Button

|



fetch("data.json")

|



Receive Response

|



Is response.ok ?

Yes	No
▼	▼

response.json() Error

▼	▼
---	---

Display Data Display Error

index.html

```
<!DOCTYPE html>
<html>
<head>
<title>XMLHttpRequest Example</title>
</head>
<body>

<h2>XMLHttpRequest Example</h2>

<button onclick="loadData()">Load</button>

<p id="demo"></p>

<script>

function loadData(){

let xhr=new XMLHttpRequest();

xhr.onreadystatechange=function(){

if(this.readyState===4 && this.status===200){

let obj=JSON.parse(this.responseText);

document.getElementById("demo").innerHTML=

obj.name+"<br>" +obj.course;
```

```

}

};

xhr.open("GET","data.json",true);

xhr.send();

}

</script>

</body>
</html>

```

Output



AJAX vs Fetch API

AJAX (XMLHttpRequest)	Fetch API
Older	Modern
More code	Less code
Callback based	Promise based

AJAX (XMLHttpRequest)	Fetch API
Complex	Easy
Used in old applications	Used in modern applications

How to Execute in Visual Studio Code

Step 1

Create a folder

AJAX

Step 2

Create

index.html

script.js

data.json

Step 3

Install

Live Server Extension

Extensions →

Search

Live Server

Install

Step 4

Open

index.html

Right Click

Open with Live Server

Step 5

Click

Load Data

Output appears without refreshing the webpage.

Folder Structure

AJAX

|

|— index.html

|— script.js

|— data.json